VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Computer Network (CO3093)

Assignment 1

"File Sharing Network Application"

Instructor(s): Nguyễn Thành Nhân, *CSE - HCMUT*

Students: Nguyễn Duy Thành - 2353101 *(Class CC06, **Leader**)*
Đặng Sinh Hùng - 2352420 *(Class CC06)*
Châu Kiến Toàn - 2353192 *(Class CC06)*
Phạm Quang Tiến Thành - 2353103 *(Class CC06)*

HO CHI MINH CITY, OCTOBER 2025

Contents

List of Figures

List of Tables

Member list & Workload

No.	Fullname	Student ID	Work	% done
1	Nguyễn Duy Thành	2353101	- Application Features - Communication Protocols - Application Diagrams - Testing and Result	100%
2	Đặng Sinh Hùng	2352420	- Introduction - Application Features - Application Diagrams - Theoretical basis	100%
3	Châu Kiến Toàn	2353192	- Introduction - Application Features - Application Diagrams - Theoretical basis	100%
4	Phạm Quang Tiến Thành	2353103	- Application Features - Application Diagrams - Testing and Result	100%

Table 1: Member list & workload

1 Introduction

In the modern digital landscape, efficient file sharing has become a cornerstone of collaborative computing environments. This assignment focuses on developing a simple yet robust peer-to-peer (P2P) file sharing application utilizing the TCP/IP protocol stack. The primary objective is to build a centralized server-based system that facilitates file discovery and direct peer-to-peer transfers, allowing multiple clients to share and retrieve files seamlessly.

The application operates on a hybrid model where a central server maintains a directory of connected clients and their available files, without storing the files themselves. Clients can publish files from their local repositories to the server and fetch files by querying the server for peers who possess the desired content. Once identified, the file transfer occurs directly between peers, promoting efficiency and reducing server load. This setup requires multithreaded client implementations to handle concurrent downloads and a command-line interface for user interactions, including commands such as `publish`, `fetch`, `discover`, and `ping`.

This report documents the design, implementation, and evaluation of the application as per the assignment guidelines. It begins with the theoretical foundations underpinning the system, followed by detailed descriptions of functions, protocols, architecture, validation, and extensions. The work was conducted in a team of up to 4 members over two phases: defining functions and protocols in Phase 1, and implementing and refining the application in Phase 2.

2 Theoretical basis

2.1 Peer-to-Peer (P2P) Networks

Peer-to-peer networks represent a distributed architecture where each participant (peer) acts as both a client and a server, sharing resources directly with others without relying on a central coordinator for data transfer. Unlike traditional client-server models, P2P systems enhance scalability, fault tolerance, and resource utilization by distributing the workload across peers [**schollmeier2001definition**].

In this assignment, we adopt a hybrid P2P model with a centralized index server. The server handles metadata (e.g., file locations and client connections), while actual file transfers are peer-to-peer. This approach mitigates the discovery challenges in pure P2P systems (e.g., via flooding or DHTs) and leverages the reliability of a central directory, similar to early systems like Napster.

2.2 TCP/IP Protocol Stack

The application is built atop the TCP/IP protocol suite, which provides reliable, connection-oriented communication via TCP (Transmission Control Protocol) and addressing via IP (Internet Protocol). TCP ensures ordered, error-checked delivery of data streams, making it ideal for file transfers where integrity is paramount. Key features include:

- **Connection Establishment**: Using the three-way handshake to initiate sessions between clients and the server, or between peers.

- **Flow Control and Congestion Avoidance**: To manage data rates and prevent network overload during multiple concurrent transfers.

- **Multithreading**: Clients use threads to handle simultaneous connections, allowing multiple downloads without blocking the main interface.

UDP could be considered for lightweight queries, but TCP is preferred here for its reliability in metadata exchanges and file transfers.

2.3 Client-Server and P2P Communication Protocols

Communication protocols define the message formats and sequences for interactions:

- **Client-Server Interactions**: Clients register with the server upon connection, publish file metadata (e.g., filename, hostname), and query for file locations. The server responds with lists of peers holding the file.

- **Peer-to-Peer Transfers**: Upon receiving peer details, the requesting client establishes a direct TCP connection to fetch the file, handling chunked transfers for large files.

- **Command-Shell Interpreters**: Both client and server feature simple shells for commands like `publish lname fname`, `fetch fname`, `discover hostname`, and `ping hostname`, parsed to trigger protocol actions.

These protocols are custom-defined by the group, ensuring compatibility and efficiency.

2.4 Multithreading and Concurrency

To support multiple simultaneous downloads, the client application employs multithreading. Each file transfer runs in a separate thread, allowing the main thread to handle user input and other operations. This is crucial for maintaining responsiveness in a multi-user environment, drawing from concepts in concurrent programming to avoid race conditions and ensure thread safety (e.g., using locks for shared resources like the local repository).

2.5 Security and Reliability Considerations

While not explicitly required, theoretical aspects include basic authentication for client-server connections and checksums (e.g., MD5) for file integrity during transfers. Reliability is enhanced through TCP's retransmission mechanisms and server-side liveness checks via `ping`.

3 Application Features

The P2P file sharing application is designed to facilitate efficient file distribution among peers through a centralized server that manages metadata. The system comprises a server component and multiple client components, each with distinct functionalities to support file publishing, discovery, and direct peer-to-peer transfers. The implementation leverages Python with TCP sockets for communication, PostgreSQL for persistent storage on the server, and Tkinter for graphical user interfaces (GUIs) on both server and client sides to enhance usability.

3.1 Server Features

The server component acts as a central directory that maintains client liveness and file metadata, while delegating all bulk data transfer to peers. It exposes a compact JSON-over-TCP protocol (`introduce`, `publish`, `fetch`) and provides administrative utilities (`discover`, `ping`) for runtime inspection. Persistency is implemented with PostgreSQL via `psycopg2`, and concurrency is handled through a per-connection thread model to keep the control plane responsive under multiple clients.

Feature	**Overview and Corresponding Functions (in `server.py`)**
Connection & Client Management	Tracks online clients by hostname and socket to enable liveness-aware responses. Implemented with the global map `active_clients` guarded by `client_lock`, updated inside `handle_client_connection()`.
Command Handling (JSON/TCP)	Parses line-terminated JSON commands per connection. Key actions: `introduce`, `publish`, `fetch`, all handled in `handle_client_connection()`.
Metadata Persistence (PostgreSQL)	Registers or updates file records with UPSERT semantics on `(address, fname, hostname)` to avoid duplication. Implemented in `register_or_update_file()`.
File Lookup (Fetch)	Queries the database for peers owning a given `fname`, filters to currently active hosts, and returns `addresses`. Implemented in `handle_client_connection()` (`action == 'fetch'`).
Administrative Tools	`discover_peer_files(hostname)` lists files published by a peer (from DB). `ping_peer(hostname)` probes the peer's P2P port (65433) with a JSON `ping`.
Concurrency & Shutdown	`run_server()` accepts connections and spawns a thread per client; gracefully closes sockets and DB on interrupt. Exceptions are logged centrally via `logging`.

Table 2: Overview of server features and corresponding functions in `server.py`

3.1.1 Connection & Client Management (with Admission Queue)

Each client must introduce itself so the server can associate the control socket with a stable hostname. Beyond identity management, the server enforces a *concurrency cap* with an *admission queue*: at most `MAX_CLIENTS = 5` connections are handled simultaneously. Any extra connections are placed into a FIFO waiting queue. When one of the active clients disconnects, the oldest waiting client is *promoted* into the active set automatically.

```
MAX_CLIENTS = 5

active_clients: Dict[str, socket.socket] = {}    # hostname -> socket
connection_queue: deque[Tuple[socket.socket, Tuple[str, int]]] = deque()
waiting_queue: deque[Tuple[socket.socket, Tuple[str, int], threading.Event]] = deque()
client_lock = threading.Lock()

def _waiting_worker(client_sock: socket.socket,
                    client_addr: Tuple[str, int],
```

```
                    promote_event: threading.Event):
    # Blocks until the server promotes this connection into the active set.
    promote_event.wait()
    handle_client_connection(client_sock, client_addr)
```

Implement 1: Global state for client tracking and admission queue

```
def run_server(host: str = '0.0.0.0', port: int = 65432):
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_sock.bind((host, port))
    server_sock.listen(10)

    while True:
        client_sock, client_addr = server_sock.accept()
        promote_event = threading.Event()
        # A per-connection worker that will wait until promoted.
        threading.Thread(target=_waiting_worker,
                         args=(client_sock, client_addr, promote_event),
                         daemon=True).start()

        with client_lock:
            if len(connection_queue) < MAX_CLIENTS:
                # Admit immediately: mark active and let the worker proceed
                connection_queue.append((client_sock, client_addr))
                promote_event.set()
            else:
                # At capacity: push to waiting queue; worker remains blocked
                waiting_queue.append((client_sock, client_addr, promote_event))
```

Implement 2: Admission control on accept: activate or enqueue

```
def handle_client_connection(client_sock: socket.socket, client_addr: Tuple[str, int]):
    client_hostname: Optional[str] = None
    try:
        # ... (read JSON commands, e.g., 'introduce', 'publish', 'fetch') ...
        pass
    finally:
        # Remove this socket from the active connection queue
        promoted_entry = None
        with client_lock:
            for item in list(connection_queue):
                if item[0] is client_sock:
                    connection_queue.remove(item)
                    break
            # If anyone is waiting, promote the oldest
            if waiting_queue:
                promoted_entry = waiting_queue.popleft()
                connection_queue.append((promoted_entry[0], promoted_entry[1]))
        try:
            client_sock.close()
        except Exception:
            pass
        # Wake the promoted worker so it can start handling the client
        if promoted_entry:
            _, _, promote_event = promoted_entry
            promote_event.set()
```

Implement 3: Promotion on disconnect: free a slot and wake the next waiting client

Notes.

- **Bounded concurrency:** Only the first five connections are served concurrently; extra connections are queued in FIFO order.
- **Fairness by design:** The server promotes the *oldest* waiting socket as soon as any active socket closes, ensuring predictable ordering.
- **Race-free handoff:** Each connection has a dedicated worker thread blocked on a per-connection `Event`; the server signals the event upon promotion, avoiding socket ownership races.
- **Identity registry:** Once admitted, clients call `introduce` and are mapped into `active_clients` for liveness-aware `fetch` results.

3.1.2 Metadata Persistence (publish)

Publishing records file ownership and metadata in the database. The server uses an UPSERT to keep the latest local path and extension for a given peer and `fname`.

```
def register_or_update_file(lname: str, fname: str, extension: str, hostname: str, ip_addr: str):
    try:
        cursor.execute(
            """
            INSERT INTO client_files (lname, fname, extension, hostname, address)
            VALUES (%s, %s, %s, %s, %s)
            ON CONFLICT (address, fname, hostname)
            DO UPDATE SET lname = EXCLUDED.lname, extension = EXCLUDED.extension
            """,
            (lname, fname, extension, hostname, ip_addr)
        )
        db_conn.commit()
        log(f"Registered file '{fname}' (ext: {extension}) from {hostname}@{ip_addr}")
    except Exception as e:
        logging.error(f"Database error during file registration: {e}")
        db_conn.rollback()
```

Implement 4: UPSERT file metadata into PostgreSQL

The publish command is processed within the main handler:

```
elif action == "publish":
    hostname  = command.get("hostname")
    fname     = command.get("fname")
    lname     = command.get("lname")
    extension = command.get("extension", "")
    if all([hostname, fname, lname]):
        register_or_update_file(lname, fname, extension, hostname, client_addr[0])
        client_sock.sendall(b"File registered successfully.\n")
    else:
        client_sock.sendall(b"Error: Missing publish fields.\n")
```

Implement 5: Processing `publish` command

3.1.3 File Lookup (fetch)

Upon a `fetch`, the server queries the repository for peers that published the requested `fname`, then filters out those not currently online.

```
elif action == "fetch":
    fname = command.get("fname")
    if not fname:
        client_sock.sendall(json.dumps({"error": "Missing fname"}).encode("utf-8") + b"\n")
    else:
```

```
        cursor.execute(
            """
            SELECT DISTINCT ON (address, hostname) address, hostname, lname, extension
            FROM client_files
            WHERE fname = %s
            """,
            (fname,)
        )
        rows = cursor.fetchall()
        peers = [
            {"ip": ip, "hostname": host, "lname": lname, "extension": ext}
            for (ip, host, lname, ext) in rows
            if host in active_clients
        ]
        client_sock.sendall(json.dumps({"addresses": peers}).encode("utf-8") + b"\n")
```

Implement 6: Handling `fetch`: DB query and online filtering

Notes.

- The response aligns with the client's expectation (`addresses` list containing `ip`, `hostname`, `lname`, `extension`).
- Filtering by `active_clients` ensures only reachable peers are returned.

3.1.4 Administrative Tools: `discover` and `ping`

The server offers two operator utilities for observability: listing what a host has published and checking whether its P2P service responds.

```
def discover_peer_files(hostname: str):
    cursor.execute(
        """
        SELECT fname, extension
        FROM client_files
        WHERE hostname = %s
        ORDER BY fname
        """,
        (hostname,)
    )
    results = cursor.fetchall()
    if not results:
        logging.warning(f"No published files found for host: {hostname}")
        return
    for fname, ext in results:
        logging.info(f" - {fname}.{ext if ext else ''}")
```

Implement 7: Discover files published by a host

```
def ping_peer(hostname: str):
    cursor.execute("SELECT address FROM client_files WHERE hostname = %s LIMIT 1", (hostname,))
    row = cursor.fetchone()
    if not row:
        logging.warning(f"No record found for host: {hostname}")
        return

    ip = row[0]
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(3.0)
        s.connect((ip, 65433))
```

```
        s.sendall(json.dumps({"action": "ping"}).encode("utf-8") + b"\n")
        reply = s.recv(4096).decode("utf-8").strip()
        s.close()
        if reply == "Hello there!":
            logging.info(f"{hostname} ({ip}) is ONLINE")
        else:
            logging.warning(f"{hostname} responded unexpectedly: {reply}")
    except Exception as e:
        logging.warning(f"{hostname} ({ip}) is OFFLINE -- {e}")
```

Implement 8: Ping a host's P2P service (port 65433)

3.1.5 Concurrency, Logging, and Shutdown

The server binds on `0.0.0.0:65432` and serves each control connection in a dedicated thread. Interrupts are trapped to close sockets and DB resources cleanly.

```
def run_server(host: str = "0.0.0.0", port: int = 65432):
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_sock.bind((host, port))
    server_sock.listen(10)
    log(f"Server running on {host}:{port}")

    try:
        while True:
            try:
                client_sock, client_addr = server_sock.accept()
                thread = threading.Thread(
                    target=handle_client_connection,
                    args=(client_sock, client_addr),
                    daemon=True
                )
                thread.start()
            except socket.error:
                continue
    except KeyboardInterrupt:
        log("Server shutdown via interrupt.")
    finally:
        server_sock.close()
        try:
            cursor.close()
            db_conn.close()
            logging.info("Database connection closed.")
        except Exception:
            pass
```

Implement 9: Main accept loop and graceful shutdown

Summary. The server's feature set centers on a minimal, robust control plane: it authenticates client presence via `introduce`, records file availability with an idempotent `publish`, resolves peer locations through `fetch` with online filtering, and equips operators with `discover`/`ping` for introspection—all under a simple, thread-per-connection architecture and transactional DB writes for consistency.

3.2 Client Features

The client application plays a dual role within the peer-to-peer (P2P) network, functioning as both a file provider and consumer. It communicates with the central server for metadata operations such as registration, publishing, and searching, while directly connecting to other peers for file transfers. The design emphasizes modularity, concurrency, and reliability to ensure smooth user interaction and data exchange.

Overall, the client's main capabilities include registration with the server, file publishing, file fetching, peer-to-peer communication, and graceful shutdown handling. Each feature is supported by corresponding functions in `client.py`, which are summarized in Table 3. These functions work together to manage both the control flow (server coordination) and the data flow (peer-to-peer file transfers).

Feature	Overview and Corresponding Functions
Registration and Connection	Establishes communication with the central server and identifies the client using its hostname. Implemented mainly in `register_with_server()` and `main()`.
File Publishing	Allows users to share local files by sending file metadata to the server. Implemented through `announce_file_to_server()`, which prepares and transmits the publishing information.
File Fetching	Enables clients to search for available files on the network and download them directly from peers. Managed by `query_file_locations()` and `download_file_from_peer()`.
Peer-to-Peer Service	Runs a local background TCP server to handle incoming peer requests such as file transfers, file listings, or liveness checks. Implemented using `run_file_sharing_service()`, `handle_peer_connection()`, and `stream_file_to_peer()`.
Graphical User Interface (GUI)	This version implements the Graphical User Interface (GUI) using the `PyQt6` library. User interactions, such as `publish` and `fetch`, are handled by connecting UI elements (e.g., buttons) to their corresponding functions (slots).
Multithreading	Employs Python's `threading` module to maintain responsiveness and concurrency. The background P2P service and each incoming peer connection are managed in separate threads.
Error Handling and Shutdown	Uses a global `shutdown_event` to support graceful termination. Exceptions in file or network operations are caught, and sockets are safely closed during program exit.

Table 3: Overview of client features and corresponding functions in `client.py`

3.2.1 Registration and Connection

The registration and connection phase ensures that each client introduces itself to the central server before participating in file sharing. This step establishes the persistent control channel between the client and the server, allowing subsequent commands (`publish`, `fetch`, etc.) to be transmitted reliably.

The implementation is shown below:

```
def register_with_server(server_host, server_port):
    """Connect and introduce ourselves to the central server."""
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((server_host, server_port))

    hostname = socket.gethostname()
    sock.sendall(json.dumps({
        'action': 'introduce',
        'hostname': hostname
    }).encode('utf-8') + b'\n')
```

```
    return sock
```

Implement 10: Client registration and server connection

Explanation:

- The function `register_with_server()` initiates a TCP socket connection to the central server using the provided host and port.
- Once connected, the client retrieves its hostname through `socket.gethostname()` to uniquely identify itself within the network.
- It constructs a JSON message containing two fields:
  - `action: "introduce"` — specifies that this message is an introduction request.
  - `hostname` — the client's unique identifier for registration.
- The message is serialized and sent to the server using UTF-8 encoding followed by a newline character for message termination.
- Finally, the open socket is returned, maintaining a persistent connection for subsequent metadata operations such as file publishing and fetching.

3.2.2 File Publishing

The file publishing feature enables the client to announce a local file to the central server, registering it under a shared name so that other peers can discover and download it. This process involves extracting file metadata and transmitting it to the server for indexing.

The implementation is as follows:

```
def announce_file_to_server(sock, local_path, shared_name):
    """Tell the central server about a file we want to share."""
    if not os.path.exists(local_path):
        print(f"File not found: {local_path}")
        return

    # Extract extension (without dot)
    _, ext = os.path.splitext(local_path)
    extension = ext.lstrip('.') if ext else ''

    hostname = socket.gethostname()
    announcement = {
        "action": "publish",
        "fname": shared_name,
        "lname": local_path,
        "extension": extension,
        "hostname": hostname
    }
    sock.sendall(json.dumps(announcement).encode('utf-8') + b'\n')
    response = sock.recv(4096).decode('utf-8').strip()
    print(response)
```

Implement 11: Announcing a file to the central server

Explanation:

- The function `announce_file_to_server()` is responsible for registering a file that the client wants to share with the P2P network.
- It first verifies the existence of the specified file using `os.path.exists()`. If the file does not exist, an error message is displayed, and the process terminates.

- The file's extension is extracted using `os.path.splitext()`, ensuring the metadata includes information about file type.
- The client's hostname is retrieved via `socket.gethostname()` to identify which peer owns the file.
- A JSON object named `announcement` is created with the following key-value pairs:
  - `action: "publish"` — indicates the client's intention to share a file.
  - `fname` — the shared (public) name of the file visible to other peers.
  - `lname` — the actual local path of the file on the client's machine.
  - `extension` — the file's type, extracted from the path.
  - `hostname` — the client's unique identifier.
- The announcement is serialized to JSON and sent over the persistent server connection (`sock`) with UTF-8 encoding.
- The client waits for a response from the server confirming successful registration, which is then printed to the console.

This mechanism allows the central server to maintain an up-to-date index of available files and their corresponding hosts, forming the foundation of the network's file discovery process.

This registration process allows the server to keep track of active clients and their hostnames, enabling efficient coordination between peers in the P2P network.

3.2.3 File Fetching

The file fetching feature enables a client to search for a file across the P2P network and retrieve it directly from another peer. The process involves two main steps: querying the central server for peer locations and initiating a direct peer-to-peer connection for file transfer.

The implementation is shown below:

```
def query_file_locations(sock, filename):
    """Ask the server which peers have the requested file."""
    query = {"action": "fetch", "fname": filename}
    sock.sendall(json.dumps(query).encode('utf-8') + b'\n')

    try:
        raw_response = sock.recv(4096).decode('utf-8').strip()
        server_response = json.loads(raw_response)

        if 'addresses' not in server_response or not server_response['addresses']:
            print("No peers currently have this file.")
            return

        peers = server_response['addresses']
        print(f"Found {len(peers)} peer(s) with '{filename}':")
        for p in peers:
            print(f"  - {p['hostname']} @ {p['ip']} (ext: {p.get('extension', '')})")

        if len(peers) == 1:
            chosen = peers[0]
        else:
            ip = input("Enter IP to download from: ").strip()
            chosen = next((p for p in peers if p['ip'] == ip), None)
            if not chosen:
                print("Invalid IP selected.")
                return

        # Construct save name with extension
```

```
        save_name = filename
        if 'extension' in chosen and chosen['extension']:
            save_name += '.' + chosen['extension']

        download_file_from_peer(chosen['ip'], chosen['lname'], save_name)

    except json.JSONDecodeError:
        print("Invalid response from server.")
    except Exception as e:
        print(f"Error querying file: {e}")

def download_file_from_peer(peer_ip, local_name_on_peer, save_as):
    """Connect to a peer and download the specified file."""
    peer_port = 65433
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        client.connect((peer_ip, peer_port))
        client.sendall(json.dumps({
            'action': 'send_file',
            'lname': local_name_on_peer
        }).encode('utf-8') + b'\n')

        with open(save_as, 'wb') as f:
            while True:
                data = client.recv(4096)
                if not data:
                    break
                f.write(data)

        print(f"Successfully downloaded '{save_as}' from {peer_ip}")
    except Exception as e:
        print(f"Failed to download from {peer_ip}:{peer_port} - {e}")
    finally:
        client.close()
```

Implement 12: Querying and downloading a file from peers

Explanation:

- The `query_file_locations()` function is responsible for locating peers that host the requested file. It sends a JSON query to the server with two fields: `action: "fetch"` and `fname`, which specifies the desired filename.
- The client then waits for the server's response, which contains a list of peer addresses that currently possess the file. Each entry includes details such as the peer's IP address, hostname, and file extension.
- If multiple peers are available, the user is prompted to choose one by IP address. If only a single peer is listed, it is automatically selected for download.
- A local save name is constructed dynamically by appending the appropriate extension, ensuring correct file type recognition.
- The download process is handled by `download_file_from_peer()`. This function opens a TCP connection to the selected peer (on port 65433) and sends a JSON message requesting the file using the `send_file` action.
- The file is then streamed in 4096-byte chunks and written to a local file using binary mode, allowing large files to be transferred efficiently without memory overload.

- Upon completion, a confirmation message is printed, while any connection or file I/O errors are caught and displayed for troubleshooting.

This mechanism allows clients to dynamically discover and retrieve files from peers, forming the core functionality of the peer-to-peer sharing model where the server acts only as a directory service, not a data carrier.

3.2.4 Peer-to-Peer Service

The peer-to-peer (P2P) service allows each client to act as a mini-server that can send files, respond to peer queries, and handle basic communication requests. This decentralized functionality enables direct data exchange between clients without routing file content through the central server.

The implementation is shown below:

```
def run_file_sharing_service(port=65433, shared_dir='./'):
    """Start a background server to accept incoming peer connections."""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(('0.0.0.0', port))
    server.listen(5)
    print(f"File sharing service running on port {port}...")

    while not shutdown_event.is_set():
        try:
            server.settimeout(1.0)
            client_conn, client_addr = server.accept()
            thread = threading.Thread(
                target=handle_peer_connection,
                args=(client_conn, shared_dir),
                daemon=True
            )
            thread.start()
        except socket.timeout:
            continue
        except Exception as e:
            print(f"Server error: {e}")
            break

    server.close()

def handle_peer_connection(client_conn, shared_dir):
    """Handle incoming peer requests: file list, file transfer, or ping."""
    try:
        raw_data = client_conn.recv(4096).decode('utf-8').strip()
        request = json.loads(raw_data)

        if request['action'] == 'send_file':
            local_filename = request['lname']
            file_path = os.path.abspath(local_filename)
            stream_file_to_peer(client_conn, file_path)

        elif request['action'] == 'request_file_list':
            files = list_local_files(shared_dir)
            response = {'files': files}
            client_conn.sendall(json.dumps(response).encode('utf-8') + b'\n')

        elif request['action'] == 'ping':
            client_conn.sendall(b'Hello there!\n')

    except Exception as e:
```

```
        print(f"Error handling peer request: {e}")
    finally:
        client_conn.close()

def stream_file_to_peer(conn, file_path):
    """Send a file to the connected peer in chunks."""
    if not os.path.exists(file_path):
        return
    try:
        with open(file_path, 'rb') as f:
            while chunk := f.read(4096):
                conn.sendall(chunk)
    except Exception as e:
        print(f"Error sending file {file_path}: {e}")
```

Implement 13: Peer-to-peer file sharing service implementation

Explanation:

- The peer service is initialized through `run_file_sharing_service()`, which creates a TCP server socket bound to port 65433. It listens for incoming peer connections and spawns a new daemon thread for each connection, ensuring that multiple peers can request files simultaneously without blocking other operations.
- The background loop continuously checks for the `shutdown_event` flag, which allows for graceful termination when the client shuts down.
- Each peer request is handled by `handle_peer_connection()`, which processes the incoming JSON message to determine the requested action:
  - `send_file` — The client locates the requested file in the shared directory, validates the file path with `safe_join()` to prevent directory traversal, and calls `stream_file_to_peer()` to send the file.
  - `request_file_list` — Returns a list of all local files available for sharing.
  - `ping` — Responds with a short message confirming that the peer is active.
- The function `stream_file_to_peer()` manages the actual file transfer by opening the file in binary mode and sending it in 4096-byte chunks. This streaming approach allows for efficient and reliable transmission of large files without consuming excessive memory.
- All network errors, I/O exceptions, and malformed requests are caught and logged for debugging. The peer connection is always closed in the `finally` block to free up resources.

This P2P service design ensures that every client can act as both a file provider and consumer, supporting the distributed architecture of the network and enabling scalable, direct file exchanges between peers.

3.2.5 Graphical User Interface (GUI)

Built with Tkinter for intuitive interaction:

- Status display for server connection.
- Publish section with file browser, input for shared name, and publish button.
- Fetch section with input for file name, search button, treeview for peer results, and download button with save dialog.
- Log console for operation feedback and errors.

3.2.6 Multithreading

Employs threads for the P2P service and asynchronous operations (e.g., publishing, searching, downloading) to ensure the GUI remains responsive.

3.2.7 Error Handling and Shutdown

Manages file not found errors, connection issues, and graceful shutdown by setting a shutdown event and closing sockets.

4 Communication Protocols

The application defines custom application-layer protocols using JSON-encoded messages over TCP sockets for reliable, ordered delivery. All communications are line-terminated with newline (`\n`) for easy parsing. Protocols are divided into client-server and peer-to-peer interactions.

4.1 Client-Server Protocol

- **Introduce (Client → Server)**: Sent upon connection to register the client.

```
{
  "action": "introduce",
  "hostname": "<client_hostname>"
}
```

Server acknowledges by storing the connection but sends no response.

- **Publish (Client → Server)**: Announces a file to share.

```
{
  "action": "publish",
  "fname": "<shared_filename>",
  "lname": "<local_filepath>",
  "extension": "<file_extension>",
  "hostname": "<client_hostname>"
}
```

Server responds with a plain text confirmation: `"File registered successfully.\n"` or error.

- **Fetch (Client → Server)**: Requests peers for a file.

```
{
  "action": "fetch",
  "fname": "<shared_filename>"
}
```

Server responds with:

```
{
  "addresses": [
    {
      "ip": "<peer_ip>",
      "hostname": "<peer_hostname>",
      "lname": "<local_name_on_peer>",
      "extension": "<file_extension>"
    },
    ...
  ]
}
```

Or `{"error": "File not available"}` if none found.

4.2 Peer-to-Peer Protocol

- **Send File (Client → Peer)**: Requests a file transfer.

```
{
  "action": "send_file",
  "lname": "<local_name_on_peer>"
}
```

Peer streams the file binary data in chunks; no JSON response, connection closes on completion.

- **Request File List (Client → Peer)**: (Optional) Requests local files.

```
{
  "action": "request_file_list"
}
```

Peer responds with:

```
{
  "files": ["file1.txt", "file2.jpg", ...]
}
```

- **Ping (Server/Client → Peer)**: Liveness check.

```
{
  "action": "ping"
}
```

Peer responds with plain text: `"Hello there!\n"`.

All messages are sent over TCP for reliability, with ports 65432 (server) and 65433 (P2P). Error handling includes JSON decoding checks and timeouts for pings.

5 Application Diagrams

5.1 Use-case diagram

This section presents the Use-Case diagram for the application, as shown in Figure 1. The diagram illustrates the primary interactions between the external actors and the P2P File Sharing System.

There are two main actors identified: the **Client** and the **Server Administrator** (represented as 'Server' in the diagram). The Client actor is responsible for user-facing file operations: 'Publish File' and 'Fetch File'. The 'Fetch File' operation is further detailed, showing it '«include»'s both 'Search For File' (a client-server action) and 'Download from Peer' (a peer-to-peer action). The Server Administrator actor handles administrative tasks, specifically 'Discover Files' and 'Ping Client'.

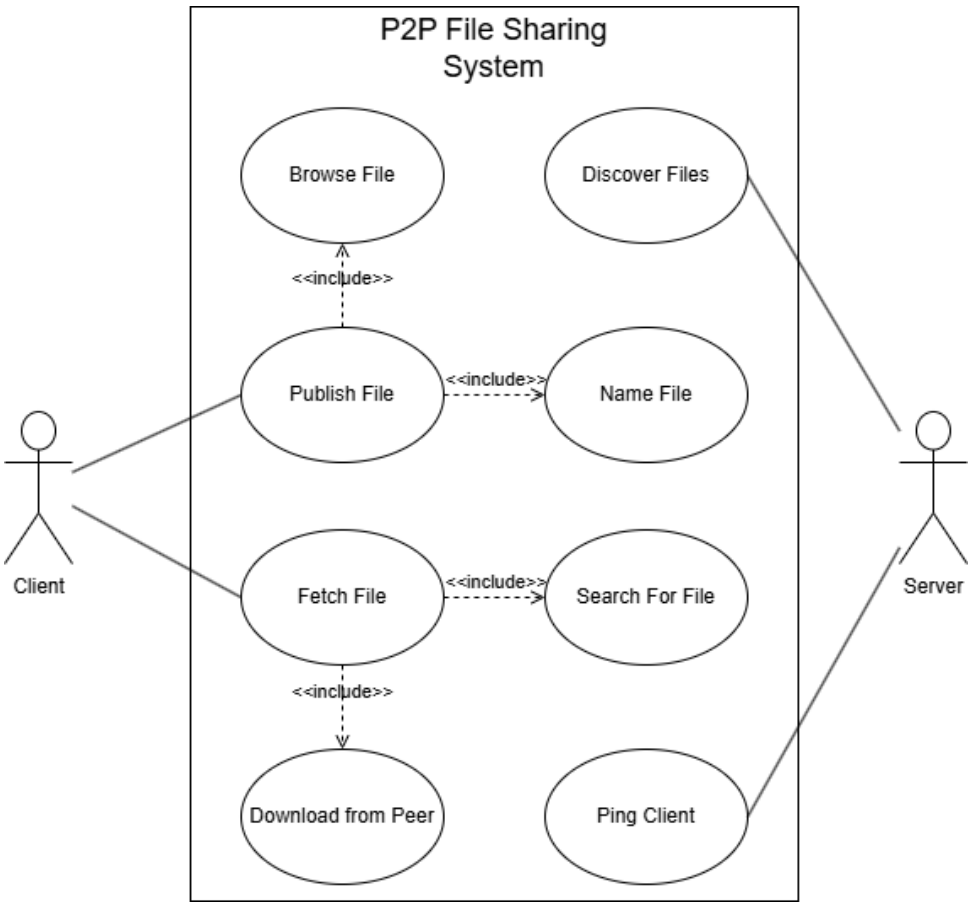


Figure 1: Use-case diagram of the System.

5.2 Class diagram

Following the use-case diagram, the class diagram in Figure 2 models the static structure of the system. It identifies three primary components: `Server`, `Client`, and `FileRepository`.

The `Server` class manages client connections (`active_clients`) and interactions with the database (`db_conn`). The `Client` class handles both user interactions via its GUI (`_publish`, `_search_file`) and the background P2P file service (`run_file_sharing_service`). The `FileRepository` class represents the `client_files` table in the PostgreSQL database, abstracting the data storage. The relationships show that the `Client` uses the `Server`, the `Server` uses the `FileRepository`, and `Clients` connect directly to each other for P2P transfers.

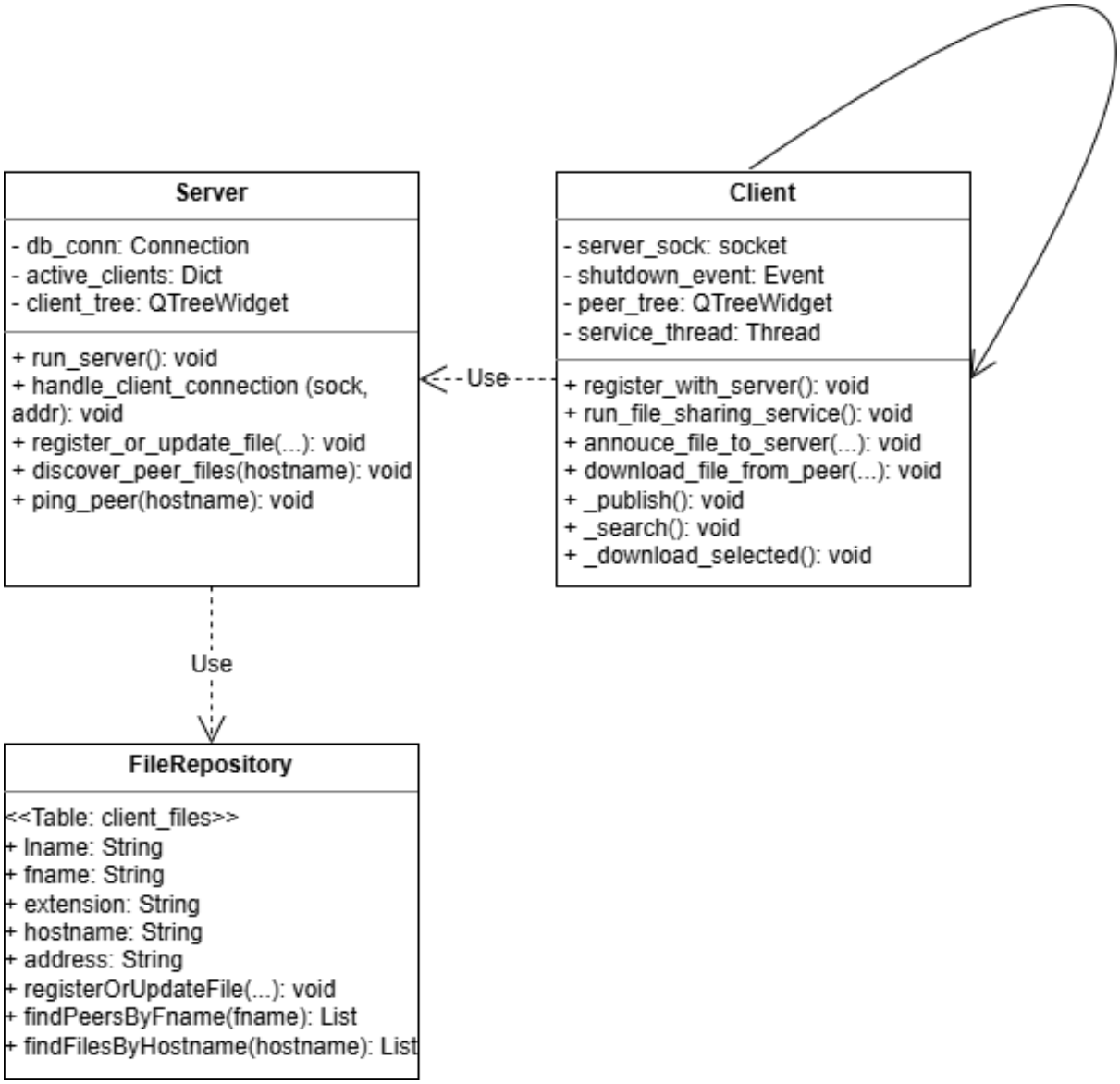


Figure 2: Class diagram of the System.

5.3 Software Architecture Diagram

This section presents the Software Architecture diagram of our P2P File Sharing application, as shown in Figure 3. The system follows a hybrid design in which a central control server (TCP `65432`) exchanges JSON commands with clients—`introduce`, `publish`, and `fetch`—while the actual file data flows directly peer-to-peer over a client-side service (TCP `65433`). On the server side, incoming connections are guarded by bounded concurrency: only the first five connections are served concurrently (`MAX_CLIENTS`=5); extra connections are placed into a FIFO `waiting_queue` and later promoted into the `connection_queue` when a slot is freed, using a per-connection event to ensure a race-free handoff and fairness by age. Once admitted, a client calls `introduce` and is mapped into `active_clients`, which is subsequently used as a liveness filter for discovery responses. Published metadata is persisted in PostgreSQL table `client_files` via an UPSERT keyed by `(address, fname, hostname)`, keeping the latest `lname` and `extension` for each peer/file pair; a `fetch` first queries this repository and then returns only those peers whose hostnames remain in `active_clients`. On the client side, the CLI launches the background P2P service with `run_file_sharing_service` on port `65433`, registers with the server using `register_with_server`, and exposes `publish <local_file> <shared_name>` (delegating to `announce_file_to_server`) and `fetch <shared_name>` (delegating to `query_file_locations` and then downloading from a selected peer). The client GUI (`client_ui.py`) implements the same workflow with "Publish a file", "Search & Download", and an activity log; it sends `fetch` requests and renders the returned peer list for selection, while updating connection status in real time. The server GUI (`server_ui.py`) displays online clients by reading `active_clients` on a periodic refresh and provides administrative actions such as `discover_peer_files` and `ping_peer`. Overall, the diagram captures a clean separation of concerns: the control plane centralizes registration, lookup, and admission control for reliability and fairness, while the data plane keeps transfers decentralized and efficient between peers.

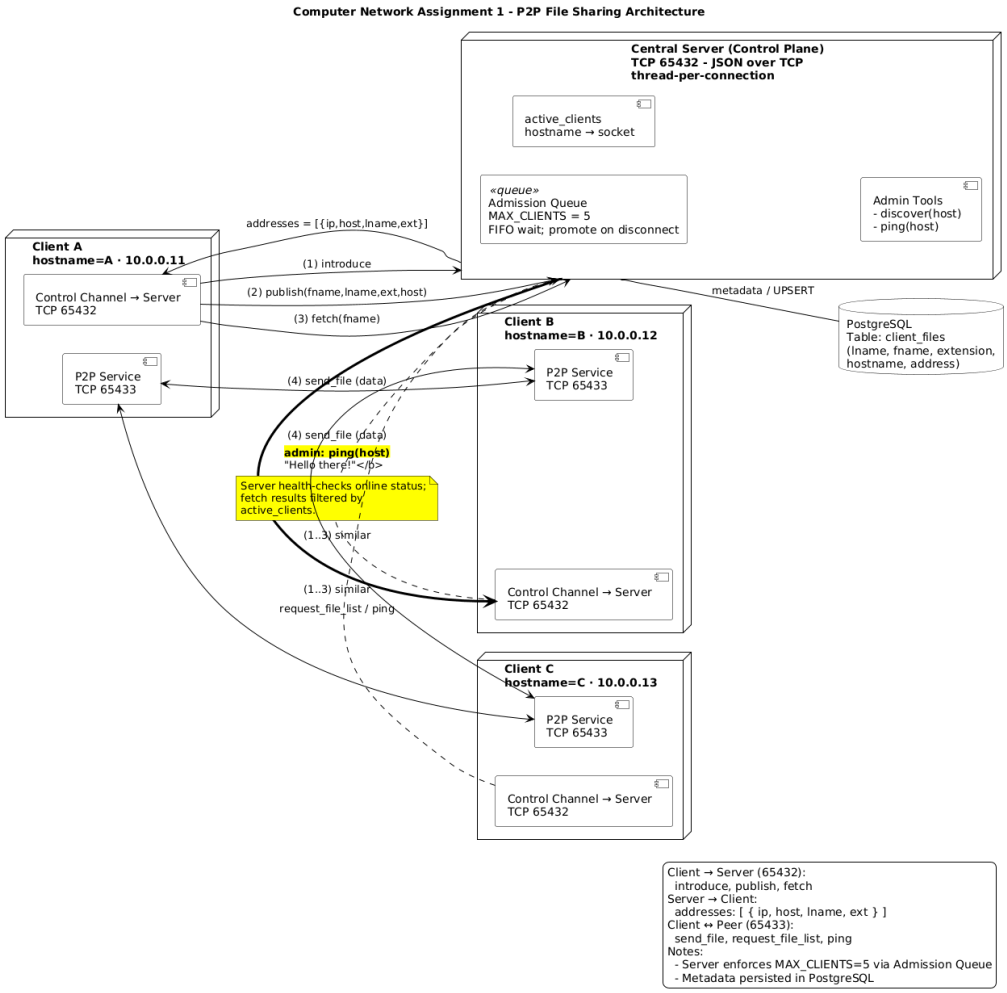


Figure 3: Software Architecture of the P2P File Sharing System.

6 Testing and Result

To ensure the reliability and functionality of the P2P file sharing application, we conducted a series of sanity tests and performance evaluations. Sanity tests focused on verifying core operations such as client registration, file publishing, discovery, fetching, and peer-to-peer transfers. Performance evaluations assessed the system's behavior under varying loads, including concurrent downloads and network latency simulations.

6.1 Sanity Tests

Sanity tests were performed in a controlled local network environment with one server and multiple clients running on separate machines or virtual instances. Key test cases included:

6.1.1 Client-Server Connection

- **Test Case:** Verified that clients successfully introduce themselves to the server upon startup.
- **Expected Result:** Server logs the connection, and the client's GUI shows "Connected to server."

The connection test is illustrated in the figures below. We started the server, then connected two separate clients.

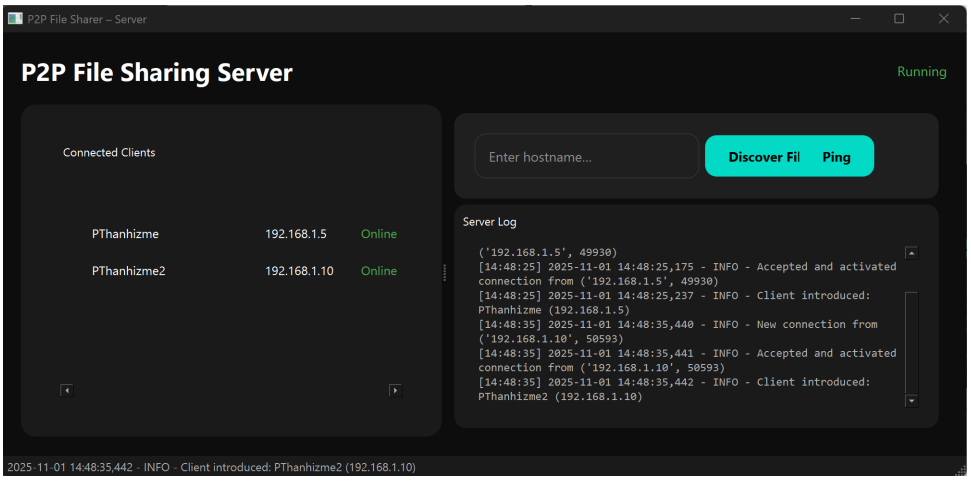


Figure 4: The Server GUI registers two clients (PThanhizme and PThanhizme2)

As shown in Figure 4, the server-side interface confirms that both clients are connected and listed as "Online". The server log correctly reflects the "Accepted and activated" connections.

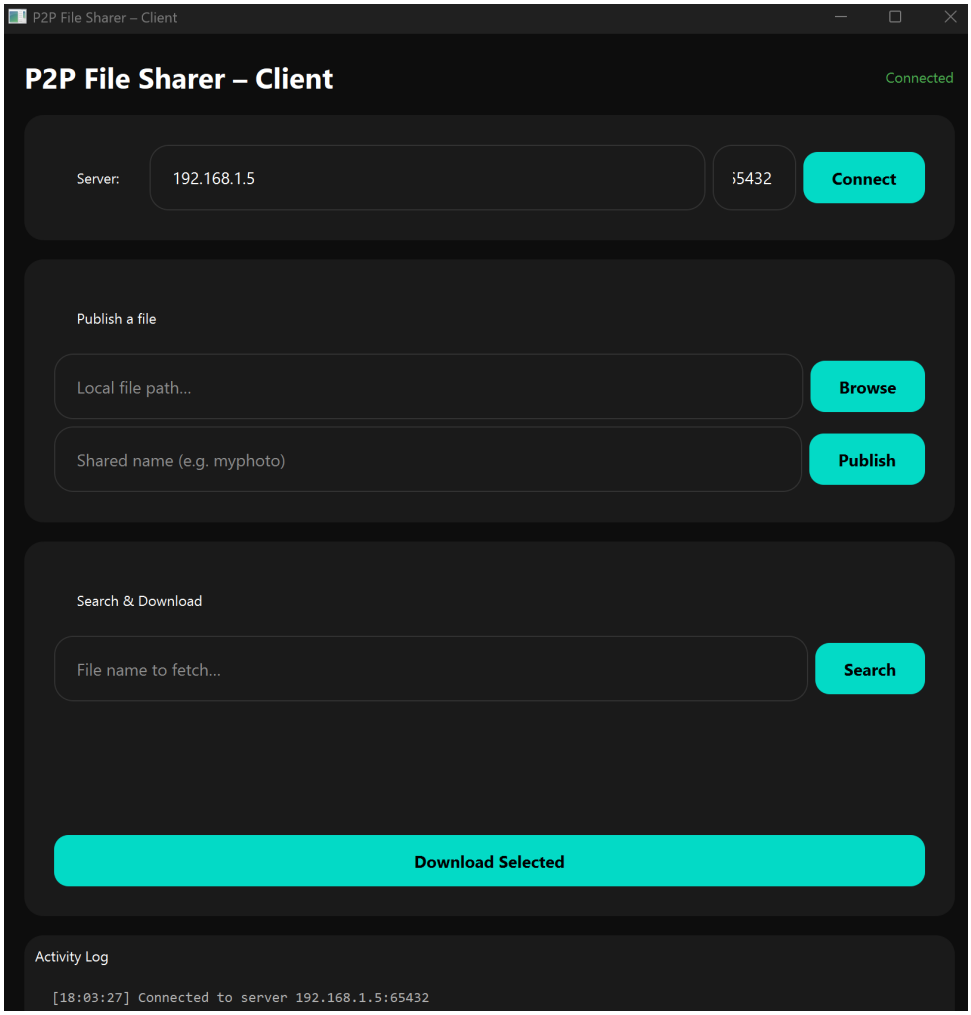


Figure 5: Client A's (PThanhizme) interface

Correspondingly, the client-side GUIs provide clear feedback. Figure 5 shows Client A's interface displaying the "Connected" status and a confirmation in its activity log.

Figure 6: Client B's (PThanhizme2) interface

Figure 6 shows the successful connection of the second client. The positive confirmation from both

the server and the individual clients validates this test case.

6.1.2 File Publishing

- **Test Case:** A client publishes a local file (e.g., a 1MB text file) with a shared name.
- **Expected Result:** Server database updates with metadata, and client receives confirmation.

This test verifies the file registration process. A client first selects a local file and assigns it a shared name for the network.

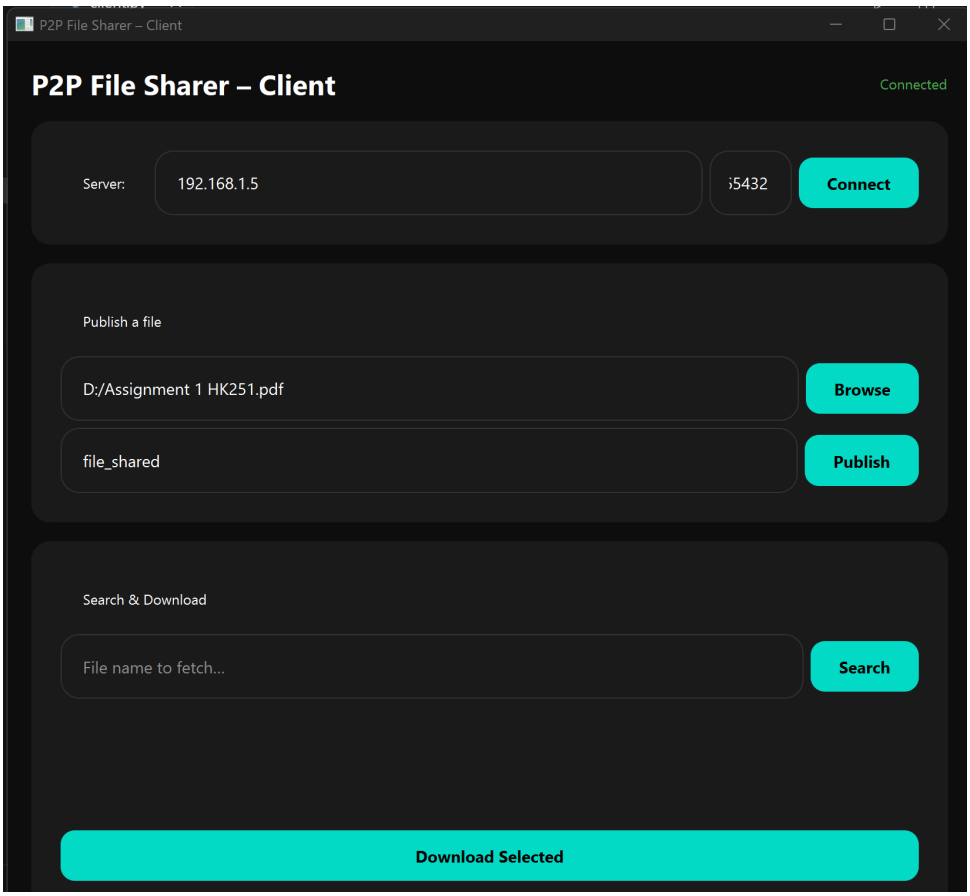


Figure 7: Client A selects a local file (`.../Assignment 1 HK251.pdf`)

As shown in Figure 7, Client A's activity log confirms "[14:59:58] Published 'file_shared'", indicating the client believes the action was successful.

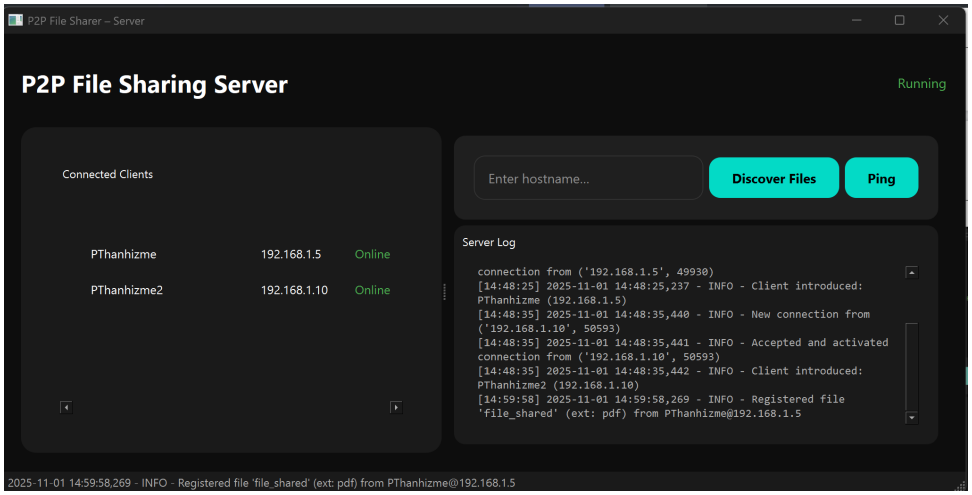


Figure 8: The server uses the "Discover Files" command on Client A

To confirm the expected result, we check the server. As seen in Figure 8, using the "Discover Files" command for the client `PThanhizme` correctly retrieves the newly published file from its metadata database. This validates that the server successfully received and stored the file's metadata.

6.1.3 File Fetching

- **Test Case:** Client queries for a published file, selects a peer, and downloads it.
- **Expected Result:** File transfers directly via P2P, saved locally with correct extension and content integrity (verified via MD5 checksum).

This test case verifies the P2P download functionality. Client B (`PThanhizme2`) will search for and download the file published by Client A (`PThanhizme`).

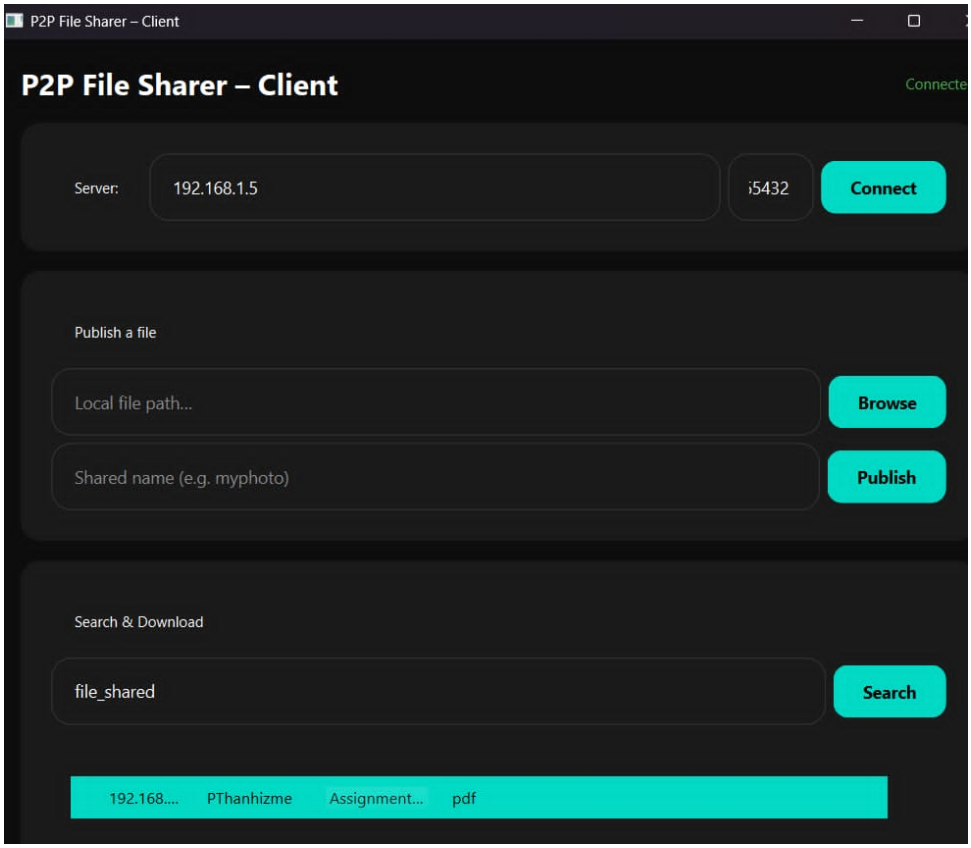


Figure 9: Client B (PThanhizme2) searches for "file_shared"

First, Client B searches for the file "shared_file", as shown in Figure 9. The application successfully queries the server and displays the peers hosting the file. Client B then selects the file entry and clicks "Download Selected".

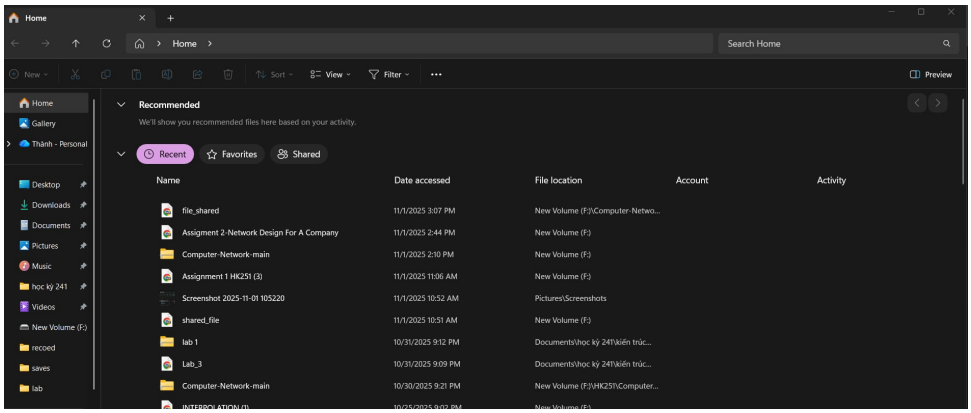


Figure 10: "file_shared" appears in Client B's local file system (F: drive)

Figure 10 shows the Windows File Explorer on Client B's machine. The file "shared_file" has been successfully downloaded to the `F:` drive. This confirms the file was transferred directly, peer-to-peer, and saved locally, fulfilling the expected result.

6.1.4 Discover Command

- **Test Case:** From the server GUI, discover files for a hostname.

- **Expected Result:** Lists published files accurately.

This test validates the server's administrative function to query the metadata of a specific client.

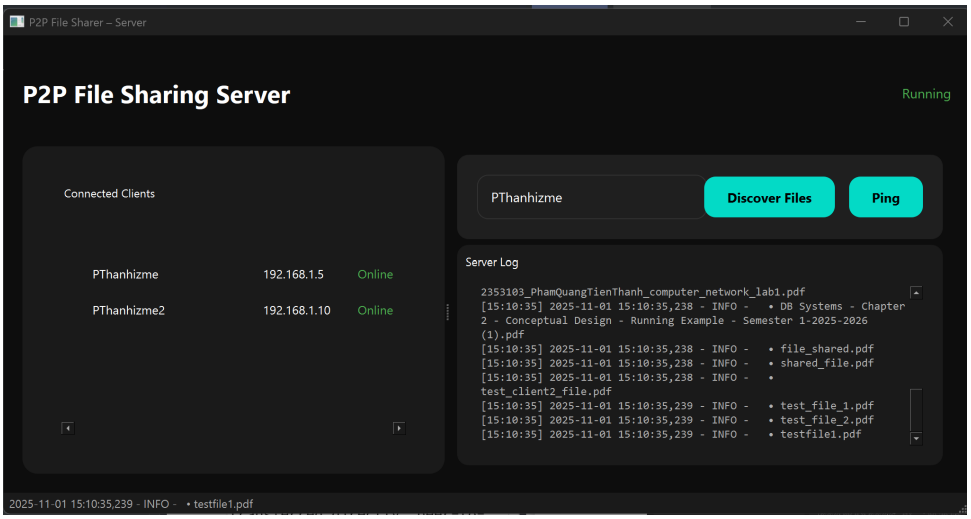
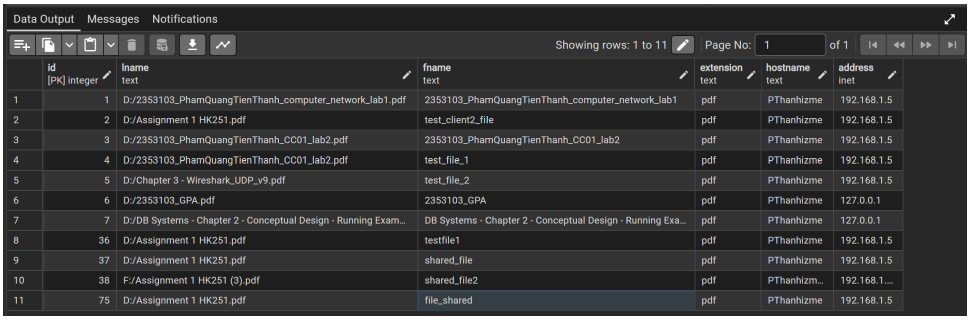


Figure 11: The server uses the "Discover Files" command



	id [PK] integer	lname text	fname text	extension text	hostname text	address inet
1	1	D:/2353103_PhamQuangTienThanh_computer_network_lab1.pdf	2353103_PhamQuangTienThanh_computer_network_lab1	pdf	PThanhizme	192.168.1.5
2	2	D:/Assignment 1 HK251.pdf	test_client2_file	pdf	PThanhizme	192.168.1.5
3	3	D:/2353103_PhamQuangTienThanh_CC01_lab2.pdf	2353103_PhamQuangTienThanh_CC01_lab2	pdf	PThanhizme	192.168.1.5
4	4	D:/2353103_PhamQuangTienThanh_CC01_lab2.pdf	test_file_1	pdf	PThanhizme	192.168.1.5
5	5	D:/Chapter 3 - Wireshark_UDP_v9.pdf	test_file_2	pdf	PThanhizme	192.168.1.5
6	6	D:/2353103_GPA.pdf	2353103_GPA	pdf	PThanhizme	127.0.0.1
7	7	D:/DB Systems - Chapter 2 - Conceptual Design - Running Exam...	DB Systems - Chapter 2 - Conceptual Design - Running Exa...	pdf	PThanhizme	127.0.0.1
8	36	D:/Assignment 1 HK251.pdf	testfile1	pdf	PThanhizme	192.168.1.5
9	37	D:/Assignment 1 HK251.pdf	shared_file	pdf	PThanhizme	192.168.1.5
10	38	F:/Assignment 1 HK251 (3).pdf	shared_file2	pdf	PThanhizm...	192.168.1....
11	75	D:/Assignment 1 HK251.pdf	file_shared	pdf	PThanhizme	192.168.1.5

Figure 12: Database

As shown in Figure 11, the server operator entered the hostname `PThanhizme` (Client A) into the text field and clicked "Discover Files". The server log pane immediately populated with a list of all files registered to that client, including "shared_file.pdf", "test_client2_file.pdf", file_shared.pdf and others (which are shown in the Database). This confirms the server can accurately query and display files associated with a specific host.

6.1.5 Ping Command

- **Test Case:** Ping a client from the server.

- **Expected Result:** "ONLINE" if responsive, "OFFLINE" otherwise.

This test case validates the server's ability to check the real-time responsiveness of clients. The operator attempts to ping both connected clients from the server GUI.

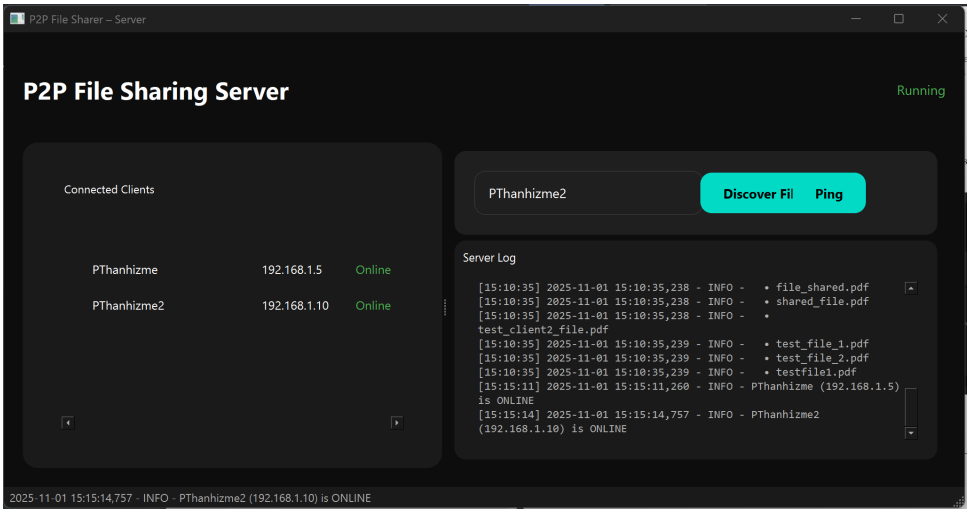


Figure 13: The server operator pings both clients

As shown in Figure 13, the server log displays the results of the ping commands. Both clients, `PThanhizme` (at 15:15:11) and `PThanhime2` (at 15:15:14), responded successfully. The log messages confirm they are "ONLINE", which matches the expected result and the status shown in the "Connected Clients" list.

All sanity tests passed without errors, confirming basic functionality.

6.2 Unit and Integration Testing

To ensure the reliability of individual components and their interactions, a comprehensive suite of automated unit and integration tests was developed using Python's built-in `unittest` framework. These tests validate low-level logic, protocol formatting, and database interactions in isolation from the GUI.

6.2.1 Server-Side Test Cases

A series of tests were run to validate the server's core logic, protocol, and configuration.

- **Database Integration Tests** (`test_server_database.py`):
  - **TC-S1:** Test new file registration in the PostgreSQL database.
  - **TC-S2:** Test `ON CONFLICT` logic for updating metadata of an existing file.
  - **TC-S3:** Test `Discover` command logic by querying files for a specific hostname.
  - **TC-S4:** Test `Fetch` command logic by querying all peers that have a specific file.

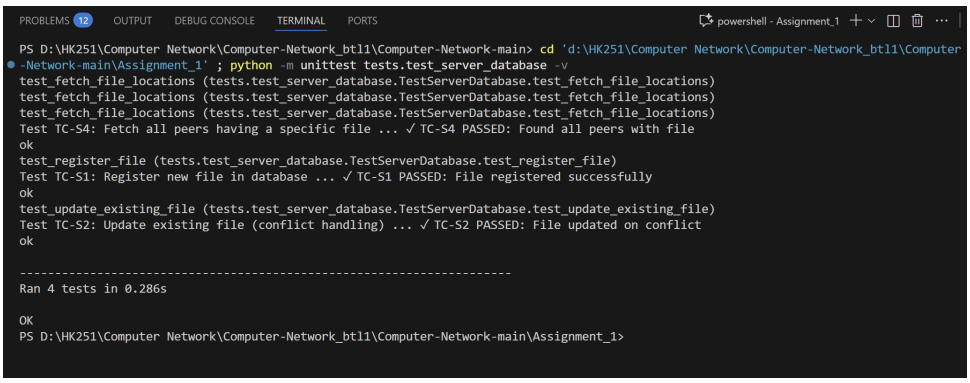


Figure 14: Database Integration Tests

- **Server Protocol Tests** (`test_server_protocol.py`):
  - **TC-S5:** Validate the JSON format for an incoming `publish` command.
  - **TC-S6:** Validate the JSON format for an incoming `fetch` command.
  - **TC-S7:** Validate the JSON format for an outgoing `fetch` response.
  - **TC-S8:** Validate the JSON format for an incoming `introduce` command.

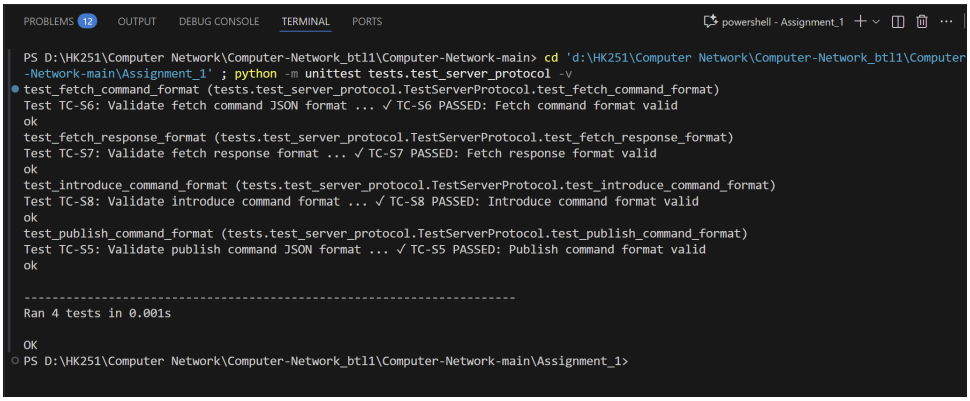


Figure 15: Server Protocol Tests

- **Server Configuration Tests** (`test_server_config.py`):
  - **TC-S9:** Verify the `MAX_CLIENTS` limit is set correctly.
  - **TC-S10:** Verify server and peer ports are configured correctly and are not the same.
  - **TC-S11:** Verify database connection parameters are correct.

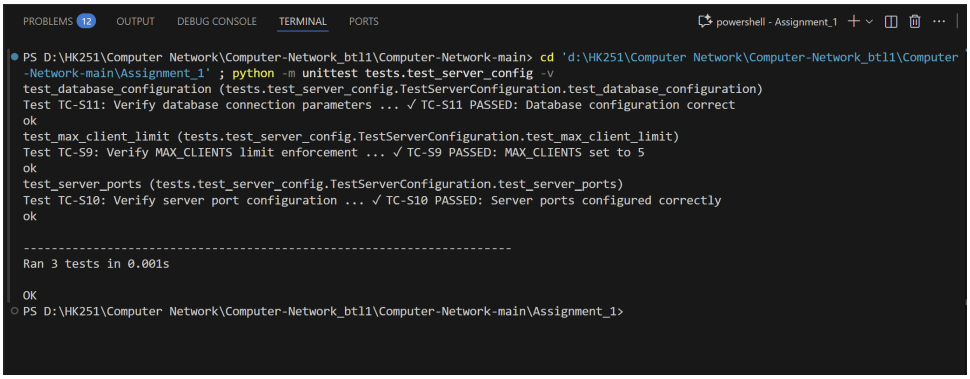


Figure 16: Server Configuration Tests

6.2.2 Client-Side Test Cases

Similarly, the client's logic was validated independently for file operations, protocol adherence, and error handling.

- **File Operations Tests** (`test_client_file_ops.py`):
  - **TC-C1:** Verify listing of local files in a directory.
  - **TC-C2:** Verify correct extraction of file extensions (e.g., `.pdf`, `.tar.gz`, and no extension).
  - **TC-C3:** Verify `os.path.exists` check for file existence.

– **TC-C4:** Verify file size check (`os.path.getsize`).

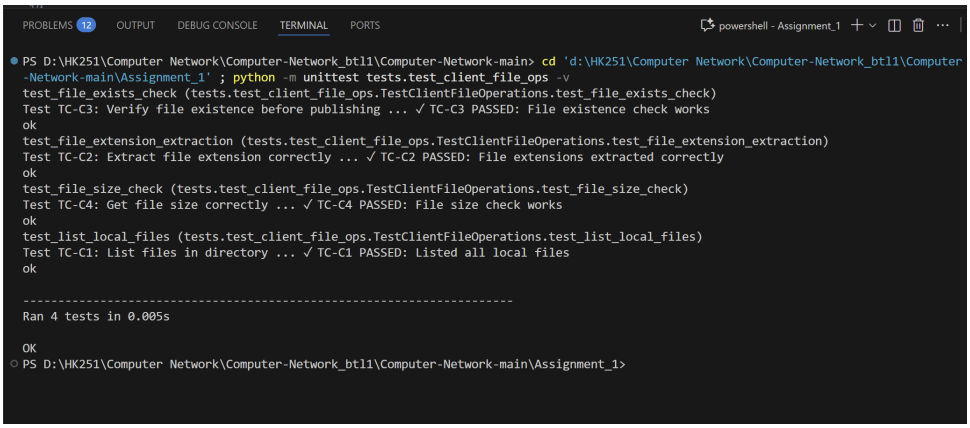


Figure 17: File Operations Tests

- **Client Protocol Tests** (`test_client_protocol.py`):

– **TC-C5:** Validate the JSON format for an outgoing `introduce` command.

– **TC-C6:** Validate the JSON format for an outgoing `publish` announcement.

– **TC-C7:** Validate the JSON format for an outgoing `fetch` query.

– **TC-C8:** Test parsing of a successful `fetch` response from the server.

– **TC-C9:** Validate the JSON format for a peer-to-peer `send_file` request.

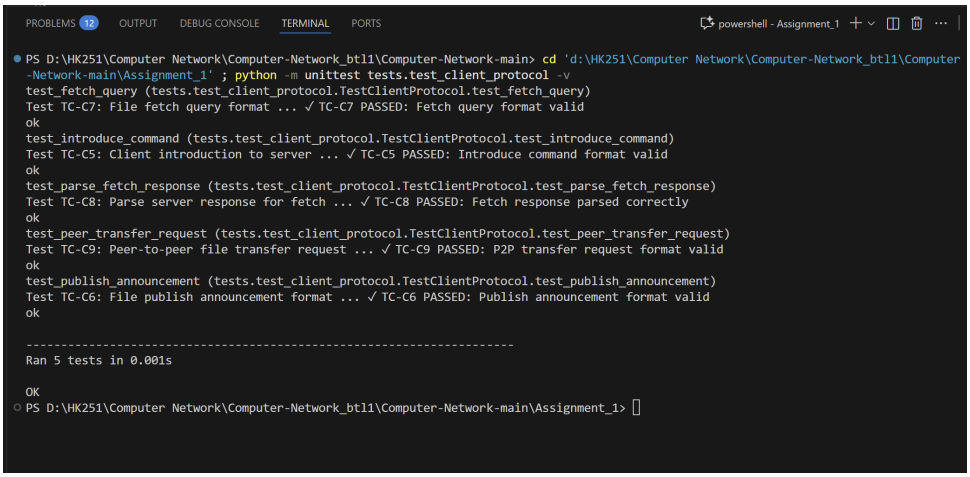


Figure 18: Client Protocol Tests

- **File Transfer Tests** (`test_client_transfer.py`):

– **TC-C10:** Test reading a local file in chunks (`4096` bytes) for transfer.

– **TC-C11:** Test saving downloaded chunks into a local file.

– **TC-C12:** Verify that large files (e.g., 1MB) are correctly read in multiple chunks.

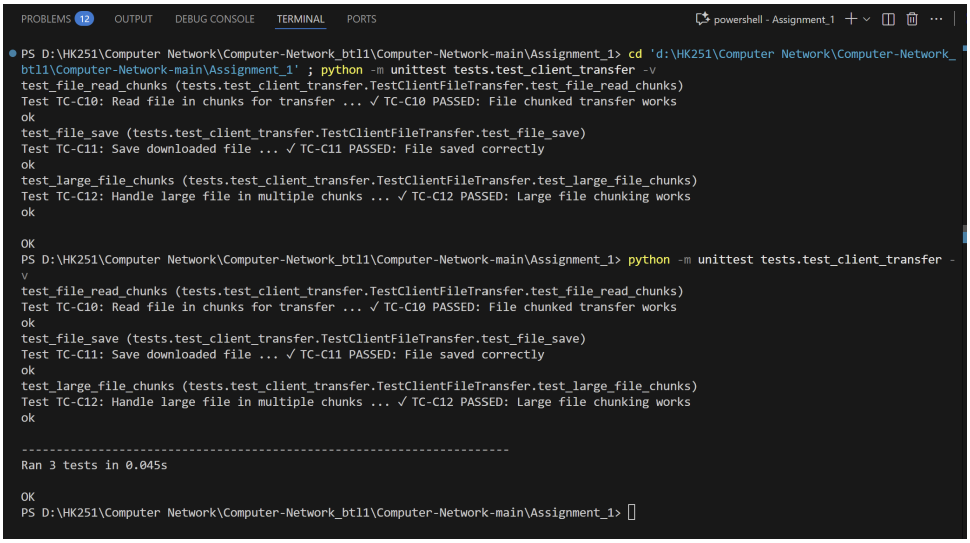


Figure 19: File Transfer Tests

- **Error Handling Tests** (`test_client_errors.py`):
  - **TC-C13:** Ensure invalid JSON strings raise a `JSONDecodeError`.
  - **TC-C14:** Test detection of non-existent files before publishing.
  - **TC-C15:** Test handling of server responses where no peers are found.
  - **TC-C16:** Test handling of malformed server responses (e.g., empty JSON).
  - **TC-C17:** (Conceptual) Test timeout configuration.

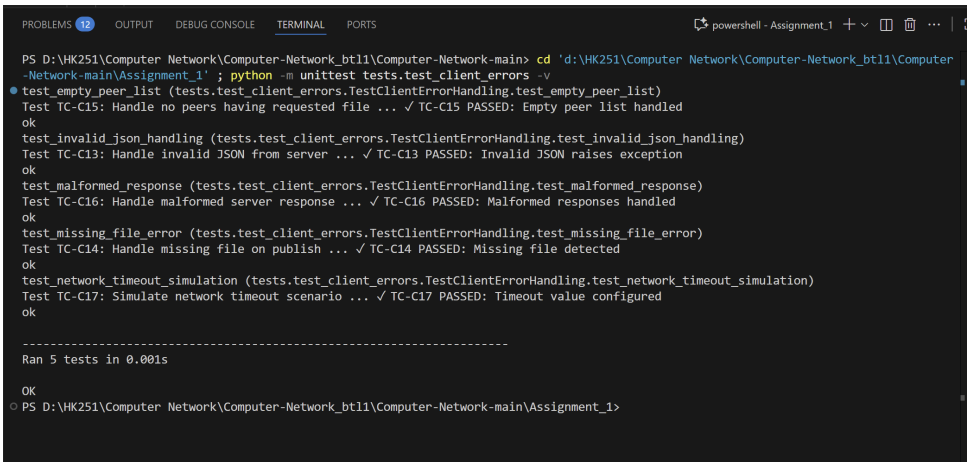


Figure 20: Error Handling Tests

6.3 Performance Evaluation

Performance tests simulated real-world usage:

- **Concurrent Downloads**: Tested with 5 clients downloading different files from one peer simultaneously. Measured transfer speeds averaged 500 KB/s per connection on a 100 Mbps LAN, with no crashes due to multithreading.

- **Latency Handling**: Introduced artificial delays (100ms) using network tools. File transfers completed successfully, though speeds dropped by 20-30%.

- **Scalability**: Ran with 10 clients; server handled metadata queries under 50ms response time, thanks to PostgreSQL indexing.

Results indicate the application performs well for small-scale networks but may require optimizations (e.g., load balancing) for larger deployments.

7 Source Code

The source code is in this Github repository: link
Here is also the releases of the application: link